

Perguntas e Gabarito de Defesa

Simulador TR1

Perguntas

1. No simulador, por que a paridade é calculada sobre o quadro inteiro, e não separadamente no final de cada byte?
2. Quando a detecção escolhida é paridade, por que o código adiciona 8 bits ao quadro, se a paridade conceitualmente precisa de apenas 1 bit?
3. Qual é a diferença prática entre paridade, checksum e CRC-32 em termos de custo no quadro e capacidade de detectar erro?
4. Por que o CRC-32 é mais forte que a paridade para detectar erros em rajada?
5. No checksum, por que a verificação correta espera que a soma do payload com o checksum recebido resulte em 0xFFFF?
6. Qual problema o enquadramento por contagem resolve e qual é a sua fragilidade principal?
7. No enquadramento por bytes, por que o transmissor precisa escapar FLAG e ESC quando eles aparecem dentro do payload?
8. No enquadramento por bits, por que o transmissor insere um zero depois de cinco bits 1 consecutivos?
9. Qual é a diferença entre detecção de erro e correção de erro no trabalho?
10. No Hamming(8,4), por que 4 bits de dados viram 8 bits transmitidos?
11. No receptor, por que o código decodifica Hamming antes de verificar o EDC?
12. No fluxo completo do simulador, onde o ruído entra: dentro da camada física ou no meio de comunicação?
13. Qual é a diferença entre modulação banda-base e modulação por portadora no código?
14. Como o ASK representa os bits no código e qual é sua principal fragilidade?
15. Como o FSK representa os bits no código e por que ele compara energia em duas frequências na demodulação?
16. Por que QPSK transmite mais informação por símbolo que ASK e FSK?

17. Por que 16-QAM transmite mais informacao por simbolo que QPSK, mas tende a ser mais sensivel a ruido?
18. O que significa dizer que QPSK e 16-QAM usam uma constelacao I/Q?
19. Para que serve a funcao de correlacao na demodulacao das modulacoes por portadora?
20. Por que o codigo usa tabelas/dicionarios de funcoes para escolher enquadramento, EDC e modulacao?

Gabarito

1. A paridade é um campo de detecção do quadro. O transmissor pega todos os bits do payload daquele quadro, conta quantos bits 1 existem e adiciona uma paridade para o bloco inteiro. Isso evita tratar cada byte isoladamente e segue a ideia da camada de enlace: proteger o quadro transmitido.

```
def adicionar_paridade_par(bits):  
    paridade = sum(bits) % 2  
    return bits + [0] * 7 + [paridade]
```

2. Porque o simulador organiza o tamanho dos campos de detecção em bytes. Conceitualmente, a paridade é só 1 bit; no código, ela é armazenada no último bit de um byte de EDC. Os outros 7 bits ficam zerados.

```
TAMANHO_EDC = {"nenhum": 0, "paridade": 1, "checksum": 2, "crc": 4}  
return bits + [0] * 7 + [paridade]
```

3. A paridade é a mais barata e simples: no simulador adiciona 1 byte por quadro, mas só 1 bit carrega a paridade. O checksum adiciona 2 bytes e detecta mais casos do que a paridade. O CRC-32 adiciona 4 bytes e é mais robusto, principalmente contra erros em rajada. Portanto, a troca é entre overhead de banda e força da detecção.
4. Porque o CRC-32 trata a mensagem como um polinômio binário e calcula um resto de divisão usando XOR e deslocamentos. Isso faz com que muitos padrões de erro alterem o resto calculado. A paridade só percebe se a quantidade de bits alterados foi ímpar; se dois bits mudarem, por exemplo, ela pode não detectar.

```
if (crc ^ bit) & 1:  
    crc = (crc >> 1) ^ POLI_CRC32_REFLETIDO  
else:  
    crc >>= 1
```

5. No checksum de complemento de 1, o transmissor envia o complemento da soma. No receptor, soma-se payload mais checksum. Se nada foi alterado de forma detectável, o resultado fecha em todos os bits 1, isto é, 0xFFFF.

```
checksum = (~soma) & 0xFFFF  
total = soma_complemento1(  
    bits_para_palavras16(payload) + bits_para_palavras16(bits[-16:])  
)  
return payload, total == 0xFFFF
```

6. O enquadramento por contagem resolve a separação dos quadros colocando, no início de cada quadro, um byte informando quantos bytes o payload possui. A fragilidade é que, se esse byte de tamanho for corrompido, o receptor pode passar a cortar o fluxo no lugar errado.

```
n_bytes = len(payload) // 8
fluxo += bytes_para_bits([n_bytes])
fluxo += payload
```

7. Porque FLAG marca inicio e fim de quadro, e ESC marca que o proximo byte foi protegido. Se esses valores aparecerem no payload sem escape, o receptor pode confundir dado com controle. O escape permite dizer: este byte parece controle, mas aqui ele e dado.

```
if byte in (FLAG_BYTE, ESC_BYTE):
    quadro.append(ESC_BYTE)
quadro.append(byte)
```

8. Porque a flag em bits e 01111110. Se o payload tiver muitos bits 1 consecutivos, ele pode formar uma sequencia parecida com a flag. Ao inserir um zero depois de cinco bits 1, o transmissor garante que a flag nao apareca acidentalmente dentro do payload.

```
if uns == 5:
    trem.append(0)
    uns = 0
```

9. Deteccao de erro apenas informa se ha indicio de erro; correcao tenta recuperar automaticamente o dado. Paridade, checksum e CRC sao mecanismos de deteccao. Hamming e mecanismo de correcao para erro simples e tambem detecta erro duplo.
10. Porque o Hamming(8,4) pega 4 bits de dados e adiciona bits de paridade em posicoes especificas. No codigo, os dados sao d1, d2, d3, d4; as paridades sao p1, p2, p4 e p0. Assim, cada bloco de 4 bits vira um bloco de 8 bits.

```
bloco = [p1, p2, d1, p4, d2, d3, d4]
p0 = sum(bloco) % 2
saida += bloco + [p0]
```

11. Porque o transmissor adiciona EDC antes de aplicar Hamming. Logo, no receptor, o inverso correto e primeiro remover/corrigir Hamming e depois verificar o EDC. Assim, o Hamming pode corrigir erro tanto no payload quanto no campo de deteccao.

```
bloco = ADICIONAR_EDC[config["deteccao"]](bloco)
if config["correcao"] == "hamming":
    bloco = codificar_hamming(bloco)

if config["correcao"] == "hamming":
    payload, info["corrigidos"], info["erro_duplo"] = decodificar_hamming(
        payload)
payload, info["edc_ok"] = VERIFICAR_EDC[config["deteccao"]](payload)
```

12. O ruído entra no meio de comunicação, não dentro da camada física. A camada física gera o sinal modulado e depois demodula o sinal recebido. O meio simula a imperfeição do canal, somando ruído gaussiano às amostras do sinal.

```
ruido = random.gauss(media, sigma)
saida.append(amostra + ruido)
```

13. Na banda-base, os bits viram diretamente níveis ou transições de tensão, como NRZ-Polar, Manchester e Bipolar. Na portadora, os bits escolhem propriedades de uma senoide, como amplitude, frequência, fase ou ponto I/Q.

14. No ASK, bit 1 transmite uma portadora com amplitude; bit 0 transmite amplitude zero. A fragilidade é que ruído ou atenuação podem alterar a amplitude e confundir presença de onda com ausência de onda.

```
if bit == 1:
    sinal += onda(V, 0.0, CICLOS_PORTADORA)
else:
    sinal += onda(0.0, 0.0, CICLOS_PORTADORA)
```

15. No FSK, bit 0 e bit 1 usam frequências diferentes. Na demodulação, o receptor mede quanto o símbolo recebido parece com a frequência do 0 e quanto parece com a frequência do 1. A maior energia indica o bit mais provável.

```
f0, f1 = CICLOS_FSK
e0 = math.hypot(*correlacionar(simbolo, f0))
e1 = math.hypot(*correlacionar(simbolo, f1))
```

16. Porque QPSK agrupa bits de 2 em 2. Cada par escolhe um dos quatro pontos da constelação. ASK e FSK, no código, enviam 1 bit por símbolo; QPSK envia 2 bits por símbolo, aumentando a eficiência.

```
MAPA_QPSK = {
    (0, 0): (A_QPSK, A_QPSK),
    (0, 1): (-A_QPSK, A_QPSK),
    (1, 1): (-A_QPSK, -A_QPSK),
    (1, 0): (A_QPSK, -A_QPSK),
}
```

17. Porque 16-QAM agrupa bits de 4 em 4, escolhendo 2 bits para o eixo I e 2 bits para o eixo Q. Como existem 4 níveis em cada eixo, há 16 pontos possíveis. A desvantagem é que os pontos ficam mais próximos; com ruído, o ponto recebido pode cair mais perto do vizinho errado.

```
i = NIVEIS_GRAY[(bits[k], bits[k + 1])] * ESCALA_QAM
q = NIVEIS_GRAY[(bits[k + 2], bits[k + 3])] * ESCALA_QAM
```

18. Significa que cada simbolo e representado por duas componentes: I, em fase com o coseno, e Q, em quadratura com o seno. No codigo, a onda e gerada por uma combinacao dessas duas componentes.

```
amostras.append(i * math.cos(angulo) - q * math.sin(angulo))
```

19. A correlacao serve para estimar, no receptor, quais componentes I e Q estavam presentes no simbolo recebido. O codigo multiplica o sinal pelas referencias de cosseno e seno; se o simbolo estiver alinhado com uma referencia, a soma cresce. Depois, o receptor decide o ponto ou frequencia mais provavel.

```
soma_cos += amostra * math.cos(angulo)
soma_sen += amostra * math.sin(angulo)
i = 2 / N * soma_cos
q = -2 / N * soma_sen
```

20. Porque as tabelas deixam a escolha configuravel sem uma cadeia grande de if/elif. A configuracao guarda strings como "crc", "bits" ou "qpsk", e o dicionario converte essa escolha na funcao correta.

```
ADICIONAR_EDC = {
    "nenhum": lambda b: b,
    "paridade": adicionar_paridade_par,
    "checksum": adicionar_checksum,
    "crc": adicionar_crc32,
}

bloco = ADICIONAR_EDC[config["deteccao"]](bloco)
```